

Colby Crutcher

Lab 3

Secure Coding

Part 1

In this section, you'll be using pwndbg to perform some static analysis of the included binary (the C source files are not included and so commands like list will not be usable).

1. Disassemble main and state the contents of main. (print statements and function calls in proper order)

```
pwndbg> disass main
Dump of assembler code for function main:
0x00000000000012a3 <+0>:    endbr64
0x00000000000012a7 <+4>:    push    rbp
0x00000000000012a8 <+5>:    mov     rbp, rsp
0x00000000000012ab <+8>:    sub     rsp, 0x20
0x00000000000012af <+12>:   mov     DWORD PTR [rbp-0x14], edi
0x00000000000012b2 <+15>:   mov     QWORD PTR [rbp-0x20], rsi
0x00000000000012b6 <+19>:   mov     rax, QWORD PTR fs:0x28
0x00000000000012bf <+28>:   mov     QWORD PTR [rbp-0x8], rax
0x00000000000012c3 <+32>:   xor     eax, eax
0x00000000000012c5 <+34>:   lea     rdi, [rip+0xda0]          # 0x206c
0x00000000000012cc <+41>:   call    0x10a0 <puts@plt>
0x00000000000012d1 <+46>:   mov     edi, 0x3
0x00000000000012d6 <+51>:   call    0x1218 <simpleCounter>
0x00000000000012db <+56>:   lea     rdi, [rip+0xda3]          # 0x2085
0x00000000000012e2 <+63>:   mov     eax, 0x0
0x00000000000012e7 <+68>:   call    0x10c0 <printf@plt>
0x00000000000012ec <+73>:   lea     rax, [rbp-0xc]
0x00000000000012f0 <+77>:   mov     rsi, rax
0x00000000000012f3 <+80>:   lea     rdi, [rip+0xd9b]          # 0x2095
0x00000000000012fa <+87>:   mov     eax, 0x0
0x00000000000012ff <+92>:   call    0x10d0 <__isoc99_scanf@plt>
0x0000000000001304 <+97>:   mov     eax, DWORD PTR [rbp-0xc]
0x0000000000001307 <+100>:  mov     edi, eax
0x0000000000001309 <+102>:  call    0x11c9 <simpleIf>
0x000000000000130e <+107>:  mov     eax, DWORD PTR [rbp-0xc]
0x0000000000001311 <+110>:  mov     edi, eax
0x0000000000001313 <+112>:  call    0x1218 <simpleCounter>
0x0000000000001318 <+117>:  mov     eax, 0x0
0x000000000000131d <+122>:  mov     rdx, QWORD PTR [rbp-0x8]
0x0000000000001321 <+126>:  xor     rdx, QWORD PTR fs:0x28
0x000000000000132a <+135>:  je      0x1331 <main+142>
0x000000000000132c <+137>:  call    0x10b0 <__stack_chk_fail@plt>
0x0000000000001331 <+142>:  leave
0x0000000000001332 <+143>:  ret
End of assembler dump.
```

- FILL THIS OUT!!!

2. Set a breakpoint at line 23 of main and on each of the two functions being called from main.
 - I attempted to set a breakpoint at line 23 of main. Because the program was not compiled with full debugging symbols, GDB resolved the breakpoint to line 9, which is the closest valid instruction within main. The breakpoint was still successfully set at the beginning of main

```

pwndbg> info break
Num Type      Disp Enb Address      What
1 breakpoint keep y 0x00000000000012b6 in main at cybr437lab2main.c:9
2 breakpoint keep y 0x0000000000001227 in simpleCounter
at cybr437lab2.c:22
3 breakpoint keep y 0x00000000000011d8 in simpleIf at cybr437lab2.c:9
pwndbg>

```

3. Start running the program. What are the contents of the first string in main?
 - The first string is “This is a pwndbg program”

```

0x555555552b6 <main+19> mov rax,qword ptr fs:[0x28] RAX, [0x7ffff7fb4768] => 0x30b026171dba3e00
0x555555552bf <main+28> mov qword ptr [rbp - 8], rax [0x7ffff7fdda8] => 0x30b026171dba3e00
0x555555552c3 <main+32> xor eax,eax EAX => 0
0x555555552c5 <main+34> lea rdi,[rip + 0xda0] RDI => 0x55555555606c ← 'This is a pwndbg program'
0x555555552cc <main+41> call puts@plt <puts@plt>

```

4. Continue to the next breakpoint. What is the name and the value being passed into the function called?
 - This function is called simpleCounter. The value being passed into it is num = 3.

```

pwndbg> c
Continuing.
This is a pwndbg program

Breakpoint 2, simpleCounter (num=3) at cybr437lab2.c:22

```

5. What function did you just come from and what function are you stopped in?
 - We are in simpleCounter, and we came from main.
6. Disassemble this function you are stopped at and capture the output

```

pwndbg> disass simpleCounter
Dump of assembler code for function simpleCounter:
0x000055555555218 <+0>:      endbr64
0x00005555555521c <+4>:      push    rbp
0x00005555555521d <+5>:      mov     rbp, rsp
0x000055555555220 <+8>:      sub     rsp, 0x20
0x000055555555224 <+12>:     mov     DWORD PTR [rbp-0x14], edi
=> 0x000055555555227 <+15>:     mov     eax, DWORD PTR [rip+0x2de3]          # 0x5555555558010 <s>
0x00005555555522d <+21>:     mov     esi, eax
0x00005555555522f <+23>:     lea     rdi, [rip+0xe02]                    # 0x5555555556038
0x000055555555236 <+30>:     mov     eax, 0x0
0x00005555555523b <+35>:     call    0x5555555550c0 <printf@plt>
0x000055555555240 <+40>:     mov     DWORD PTR [rbp-0x4], 0x0
0x000055555555247 <+47>:     jmp     0x555555555275 <simpleCounter+93>
0x000055555555249 <+49>:     mov     eax, DWORD PTR [rip+0x2dc1]          # 0x5555555558010 <s>
0x00005555555524f <+55>:     mov     esi, eax
0x000055555555251 <+57>:     lea     rdi, [rip+0xe06]                    # 0x555555555605e
0x000055555555258 <+64>:     mov     eax, 0x0
0x00005555555525d <+69>:     call    0x5555555550c0 <printf@plt>
0x000055555555262 <+74>:     mov     eax, DWORD PTR [rip+0x2da8]          # 0x5555555558010 <s>
0x000055555555268 <+80>:     add     eax, 0x1
0x00005555555526b <+83>:     mov     DWORD PTR [rip+0x2d9f], eax          # 0x5555555558010 <s>
0x000055555555271 <+89>:     add     DWORD PTR [rbp-0x4], 0x1
0x000055555555275 <+93>:     mov     eax, DWORD PTR [rbp-0x4]
0x000055555555278 <+96>:     cmp     eax, DWORD PTR [rbp-0x14]
0x00005555555527b <+99>:     jle     0x555555555249 <simpleCounter+49>
0x00005555555527d <+101>:    mov     edi, 0xa
0x000055555555282 <+106>:    call    0x555555555090 <putchar@plt>
0x000055555555287 <+111>:    mov     eax, DWORD PTR [rip+0xddb]          # 0x5555555556068 <VAR>
0x00005555555528d <+117>:    mov     esi, eax
0x00005555555528f <+119>:    lea     rdi, [rip+0xdcc]                    # 0x5555555556062
0x000055555555296 <+126>:    mov     eax, 0x0
0x00005555555529b <+131>:    call    0x5555555550c0 <printf@plt>
0x0000555555552a0 <+136>:    nop
0x0000555555552a1 <+137>:    leave
0x0000555555552a2 <+138>:    ret
End of assembler dump.

```

7. How many print functions are in this current function you are stopped at?
Hint: There is more than one type of print statement.

- There are 4 print functions. There are 3 printf, and one putchar function.

8. Examining the disassembled code, grab the memory address of the constant from the function you're stopped at and display its location in the process image.

- The location of the constant is 0x5555555556038

```

0x00005555555522f <+23>:      lea     rdi, [rip+0xe02]                    # 0x5555555556038
0x000055555555236 <+30>:      mov     eax, 0x0
0x00005555555523b <+35>:      call    0x5555555550c0 <printf@plt>

```

9. Continue running (enter 15). What function are you currently stopped at, what is the name and the value being passed into the function you are currently stopped at?

- The function that we stopped in is simpleIf, and the value being passed in is num=15, which is what we entered in the terminal.

10. What is the value of the first string in this function you are stopped at, and what is the name and the type of the global variable in the function where you are stopped in?

Hint: notice the legend.

- The value is: “Sorry you didn’t win”, which is at 0x555555556019.

```
0x555555551ec <simpleIf+35> lea rdi, [rip + 0xe26] RDI => 0x555555556019 ← "Sorry you didn't win\n"
```

- The type of global variable in the function is a double named ‘MAX’.

```
0x555555551f8 <simpleIf+47> mov rax, qword ptr [rip + 0xe21] RAX, [MAX]
```

- What section (from the legend) is the global variable located in?

- It is located in the DATA section:

```
pwndbg> vmmmap &MAX
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
Start      End Perm  Size Offset File
0x55555557000 0x55555558000 r--p    1000 2000 /home/ccrutter/lab3/lab3_p1
0x55555558000 0x55555559000 rw-p    1000 3000 /home/ccrutter/lab3/lab3_p1 +0x20
0x55555559000 0x5555555a000 rw-p   21000 0 [heap]
```

- Disassemble this function you are stopped at and capture the output

```
pwndbg> disass simpleIf
Dump of assembler code for function simpleIf:
0x00005555551c9 <+0>: endbr64
0x00005555551cd <+4>: push rbp
0x00005555551ce <+5>: mov rbp, rsp
0x00005555551d1 <+8>: sub rsp, 0x10
0x00005555551d5 <+12>: mov DWORD PTR [rbp-0x4], edi
=> 0x00005555551d8 <+15>: cmp DWORD PTR [rbp-0x4], 0xc
0x00005555551dc <+19>: jne 0x555555551ec <simpleIf+35>
0x00005555551de <+21>: lea rdi, [rip+0xe23] # 0x555555556008
0x00005555551e5 <+28>: call 0x555555550a0 <puts@plt>
0x00005555551ea <+33>: jmp 0x555555551f8 <simpleIf+47>
0x00005555551ec <+35>: lea rdi, [rip+0xe26] # 0x555555556019
0x00005555551f3 <+42>: call 0x555555550a0 <puts@plt>
0x00005555551f8 <+47>: mov rax, QWORD PTR [rip+0xe21] # 0x555555558020 <MAX>
0x00005555551ff <+54>: movq xmm0, rax
0x0000555555204 <+59>: lea rdi, [rip+0xe24] # 0x55555555602f
0x000055555520b <+66>: mov eax, 0x1
0x0000555555210 <+71>: call 0x555555550c0 <printf@plt>
0x0000555555215 <+76>: nop
0x0000555555216 <+77>: leave
0x0000555555217 <+78>: ret
End of assembler dump.
```

- Look for the cmp instruction in the assembly from the disassembled code. What value is the variable from question 14 being compared to?

- It is being compared to 0xc, which is 12 in decimal.

```
cmp DWORD PTR [rbp-0x4], 0xc
```

- What area of the process image is the variable s located, what storage class is it? Hint: notice the value of s as it enters the function that is called twice.

```
0x55555558000 0x55555559000 rw-p    1000 3000 /home/ccrutter/lab3/lab3_p1 +0x10
```

- Display 64 addresses of the stack (NOTE: this is not backtrace).

```

pwndbg> x/64gx $rsp
0x7fffffffdd70: 0x0000000000000000      0x00000000ff7ffd000
0x7fffffffdd80: 0x00007fffffffddb0      0x000055555555530e
0x7fffffffdd90: 0x00007fffffffdded8     0x000000001f7fe5af0
0x7fffffffdda0: 0x00000000ffffde90      0x30b026171dba3e00
0x7fffffffddb0: 0x00007fffffffde50      0x00007ffff7c2a1ca
0x7fffffffddc0: 0x00007fffffffde00      0x00007fffffffdded8
0x7fffffffddd0: 0x00000000155554040     0x00005555555552a3
0x7fffffffdde0: 0x00007fffffffdded8     0x04ac7b334852c095
0x7fffffffddf0: 0x00000000000000001     0x0000000000000000
0x7fffffffde00: 0x0000000000000000      0x00007ffff7ffd000
0x7fffffffde10: 0x04ac7b334f72c095      0x04ac6b49b7f0c095
0x7fffffffde20: 0x00007fff00000000      0x0000000000000000
0x7fffffffde30: 0x0000000000000000      0x0000000000000000
0x7fffffffde40: 0x0000000000000000      0x30b026171dba3e00
0x7fffffffde50: 0x00007fffffffdeb0      0x00007ffff7c2a28b
0x7fffffffde60: 0x00007fffffffdee8      0x00007ffff7ffe2e0
0x7fffffffde70: 0x00007fff00000000      0x00005555555552a3
0x7fffffffde80: 0x0000000000000000      0x0000000000000000
0x7fffffffde90: 0x000055555555550e0     0x00007fffffffdded0
0x7fffffffdea0: 0x0000000000000000      0x0000000000000000
0x7fffffffdeb0: 0x0000000000000000      0x000055555555510e
0x7fffffffdec0: 0x00007fffffffdec8      0x0000000000000038
0x7fffffffded0: 0x00000000000000001     0x00007fffffff1b0
0x7fffffffdee0: 0x0000000000000000      0x00007fffffff1cd
0x7fffffffdef0: 0x00007fffffff1dd      0x00007fffffff1f1
0x7fffffffdf00: 0x00007fffffff20e      0x00007fffffff227
0x7fffffffdf10: 0x00007fffffff239      0x00007fffffff24e
0x7fffffffdf20: 0x00007fffffff260      0x00007fffffff2df
0x7fffffffdf30: 0x00007fffffff2e8      0x00007fffffff2fd
0x7fffffffdf40: 0x00007fffffff30e      0x00007fffffff2ea7
0x7fffffffdf50: 0x00007fffffff333      0x00007fffffff2ea53
0x7fffffffdf60: 0x00007fffffff3ae0     0x00007fffffff2eb12
pwndbg>

```

Part 2